DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

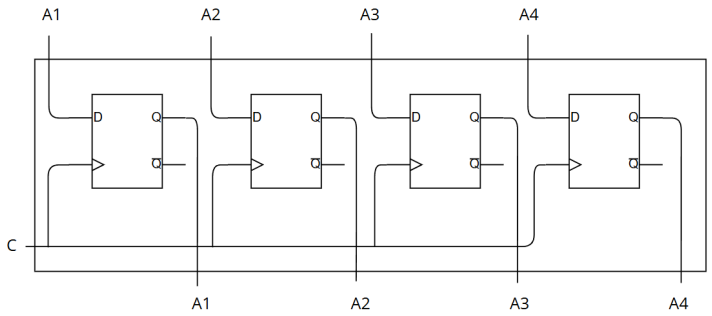
IIC2343 - Arquitectura de Computadores (I/2026)

Guía de Ejercicios: Almacenamiento

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl, Fernanda Escobar (fer_jez@uc.cl))

Pregunta 1: Modificación de registro

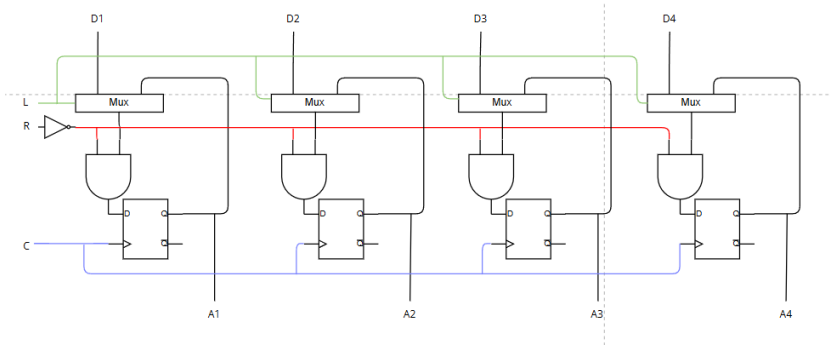
Considere el diagrama del registro de memoria de 4 bits visto en clases.



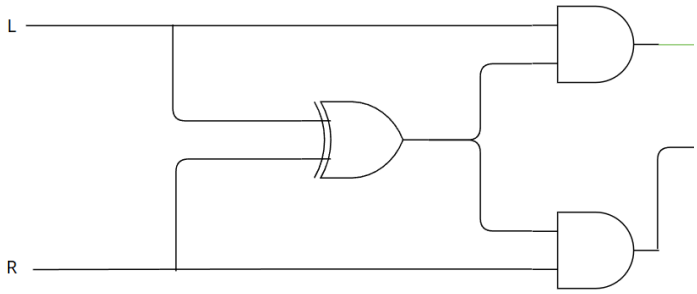
a) Modifica el diagrama interno del registro de 4 bits para que utilice las señales Load y Reset, una para cargar o no el valor de entrada (Load) y otra para limpiar el valor almacenado (Reset).
b) Modifica el diagrama anterior de forma que si tanto Load como Reset están activadas (L = R = 1), el flip-flop no cambie su valor.
c) Modifique el diagrama de la parte a) para que incluya una nueva señal E, tal que:

- Si E = 0 el registro no cambia su valor, independiente de L y R.
- Si E = 1 el registro funciona normalmente (Load y Reset operan como antes)

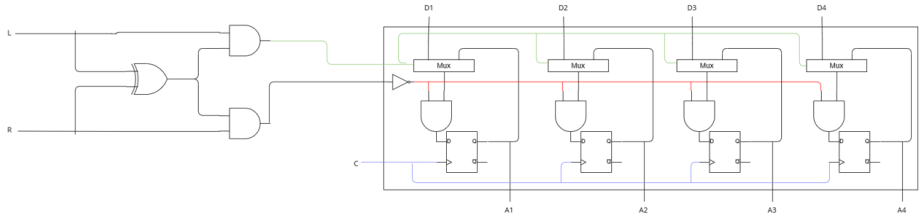
Solución: a) Agregamos un mux que mediante la señal load (L) elige si mantener el valor Q o cargar el valor nuevo D.
Asimismo se añade un AND que tiene como entrada la salida del mux y la señal R negada, de esta manera si el $R = 1$ se cargará la salida del mux al flip flop y si $R = 0$ se reseterá a 0 este valor.



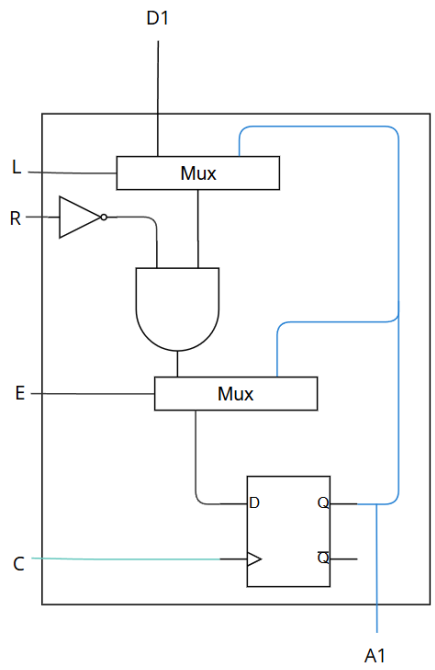
b) Notamos que, en el diagrama actual, para que el flip flop mantenga su valor debe ocurrir que L = R = 0.
De esta manera, para que el flip flop mantenga su valor cuando L = R = 1, antes de ingresar al registro, conectamos ambas señales a una compuerta XOR. Asimismo añadimos dos AND, los cuales reciben como entrada la señal (L o R) y la salida del xor.



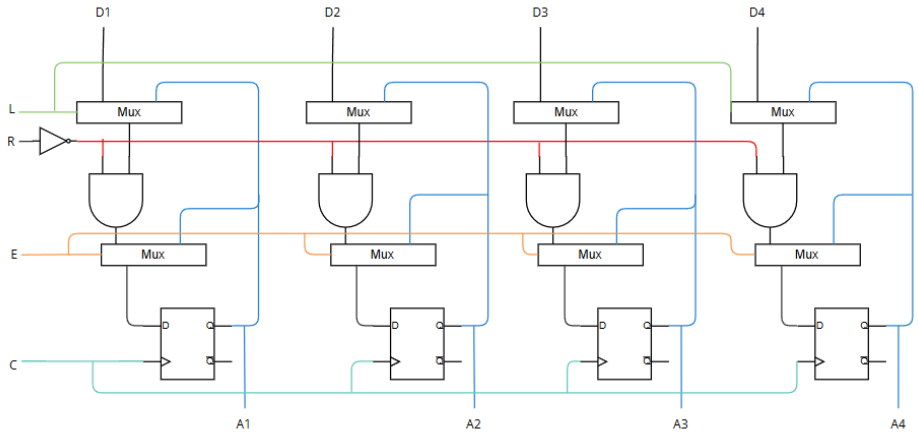
Así, cuando L y R sean iguales la salida del XOR y por consucuencia las de los AND serán 0. Esto asegura que el valor de L y R que entra al registro es L = R = 0, es decir, que el flip flop no actualiza su valor.
Finalmente, solo queda conectarlo al registro.



c) Se agrega un multiplexor adicional antes del flip-flop que selecciona entre el valor actual Q y el resultado del circuito con Load y Reset, utilizando la señal Enable como selector.

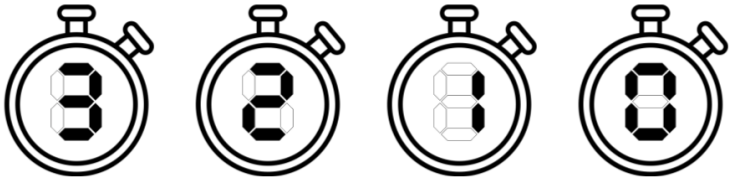


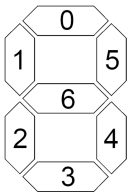
Finalmente, solo queda expandir a 4 bits.



Pregunta 2: I1 2024-2

Pronto se llevarán a cabo las DCCarreras de sacos dieciocheras y, para ahorrar recursos, le piden a usted que diseñe un *timer* para marcar el inicio de cada carrera. A continuación, una imagen de referencia del *timer* esperado y del *display* de 7 segmentos que tendrá integrado para desplegar el número:



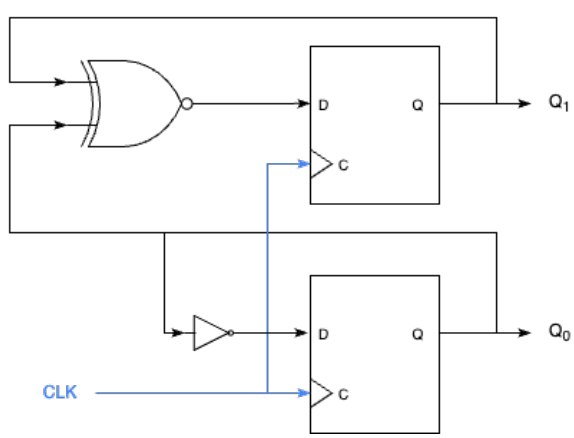


Para construir el *timer*: Diseñe un contador secuencial de 2 bits que se decrementa con cada flanco de subida de la señal de control. Este contador, además, debe recibir una señal de entrada B de 1 bit, correspondiente al valor del botón del *timer*. Si el valor de B es igual a 1 durante el flanco de subida de la señal de control (*i.e.*, el botón está presionado), entonces el contador debe actualizar su valor a 3 en vez de decrementarse en una unidad.

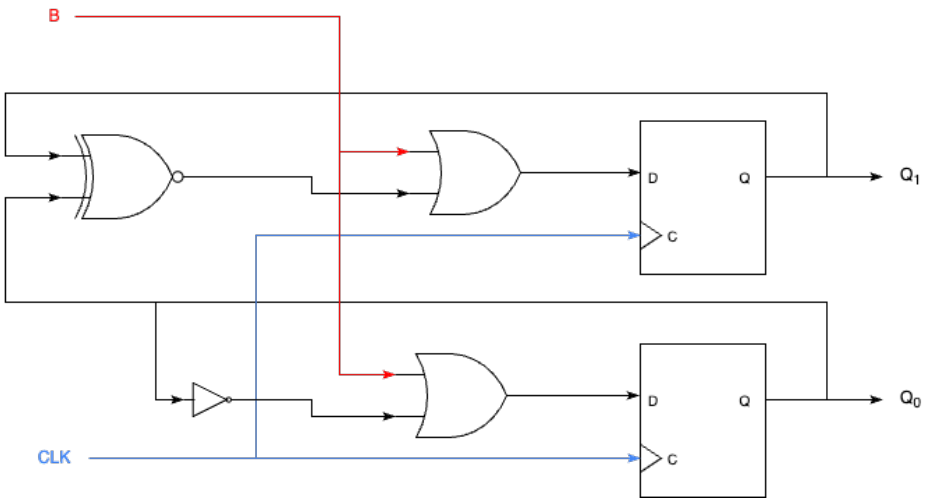
Solución: Si denotamos por $Q_1^t Q_0^t$ el estado del contador en un ciclo t de la señal de control `CLK`, y $Q_1^{t+1} Q_0^{t+1}$ su estado en el próximo ciclo, se puede elaborar la siguiente tabla de verdad:

Q_1^t	Q_0^t	Q_1^{t+1}	Q_0^{t+1}
1	1	1	0
1	0	0	1
0	1	0	0
0	0	1	1

Se deduce que $Q_0^{t+1} = \overline{Q_0^t}$, mientras que $Q_1^{t+1} = \overline{(Q_0^t \oplus Q_1^t)}$. Es decir, Q_0 será igual a `NOT` Q_0, mientras que Q_1 será igual a Q_0 `XNOR` Q_1. Haciendo uso de flip-flops y las compuertas antes señaladas, se llega al siguiente diagrama:



Ahora, falta considerar la señal B. Si $B^t = 1$, entonces $Q_1^{t+1}Q_0^{t+1} = 11$. Por lo tanto, haciendo uso de la ley de dominación, se pueden ajustar las expresiones de los estados del contador de la siguiente forma: $Q_0^{t+1} = B^t \vee \overline{Q_0^t}$; y $Q_1^{t+1} = B^t \vee \overline{(Q_0^t \oplus Q_1^t)}$. Finalmente, el siguiente diagrama representa el contador modificado conectado a la señal B:



Es importante notar que se evaluará la correctitud del contador diseñado, no que sea exactamente igual al incluido en esta pauta.

Se otorgan **1.5 ptos.** por diseñar correctamente un contador de 2 bits decremental, y **1.5 ptos.** por actualizar su valor correctamente a partir de la señal B. Por cada criterio, se otorga la mitad del puntaje si existe, como máximo, un error de implementación.

Pregunta 3: I1 2025-1

El Museo Nacional ha instalado un sistema de seguridad que utiliza sensores láser distribuidos a lo largo de un pasillo. Existen cuatro sensores, denominados L_1, L_2, L_3 y L_4, los que entregan una señal binaria de acuerdo al siguiente criterio:

- $L_i = 1$ si el haz del sensor i fue interrumpido.
- $L_i = 0$ si el haz permanece intacto.

Un estudio de seguridad reveló que una persona sospechosa, potencialmente un ladrón, interrumpe **exactamente 3** de los 4 sensores al desplazarse por el pasillo. En cambio, un visitante común puede llegar a interrumpir 1, 2 o los 4, pero **nunca exactamente 3**. Por lo tanto, se desea diseñar una alarma mediante un circuito combinacional que emita una señal binaria de salida A, tal que:

$$A = 1 \quad \text{si, y solo si, exactamente 3 de los sensores fueron interrumpidos.}$$

Adicionalmente, el museo desea que este sistema se conecte a un contador que registre la cantidad de veces que se ha activado la alarma. Como los administradores son optimistas, suponen que **no se registrarán más de 7 activaciones** en el año, por lo que ese será el valor máximo requerido. Diseñe este contador secuencial utilizando flip-flops tipo D y compuertas lógicas, de forma que se incremente en una unidad con cada flanco de subida **si, y solo si, A es igual a 1**. En su diseño se permite el uso de multiplexores, en caso de que los necesite .

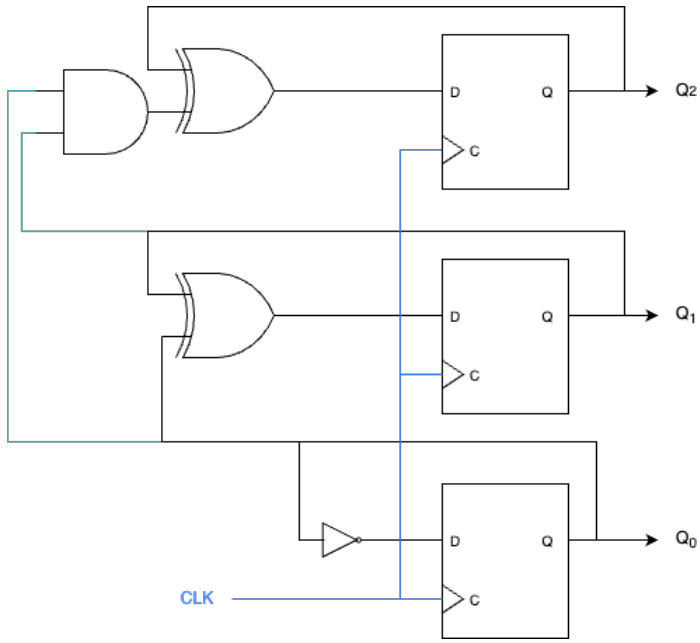
Solución: Si el contador tiene un valor máximo igual a 7, debe ser de 3 bits. Denotaremos sus salidas como $Q_2Q_1Q_0$; $Q_2^tQ_1^tQ_0^t$ como su estado en el ciclo t; y $Q_2^{t+1}Q_1^{t+1}Q_0^{t+1}$ como su estado en el siguiente ciclo. Así, se tiene la siguiente tabla de verdad:

Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Si se revisa la tabla de verdad de cada salida Q_i^{t+1}, se puede determinar que:

$$Q_0^{t+1} = \overline{Q_0^t} \qquad Q_1^{t+1} = Q_0^t \oplus Q_1^t \qquad Q_2^{t+1} = (Q_0^t \wedge Q_1^t) \oplus Q_2^t$$

Por lo tanto, se diseña el circuito a partir de las expresiones anteriores:

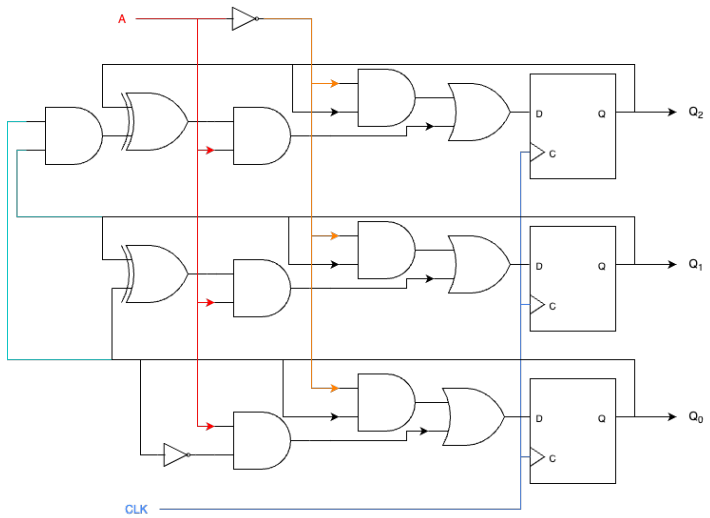


Ahora, lo anterior corresponde a un circuito secuencial que incrementa con cada flanco de subida. Se requiere, como paso final, que este se realice solo si la señal de entrada A es igual a 1. Entonces, se deben modificar las salidas de la siguiente forma:

$$Q_0^{t+1} = (A \wedge \overline{Q_0^t}) \vee (\overline{A} \wedge Q_0^t) \quad Q_1^{t+1} = (A \wedge (Q_0^t \oplus Q_1^t)) \vee (\overline{A} \wedge Q_1^t)$$

$$Q_2^{t+1} = (A \wedge ((Q_0^t \wedge Q_1^t) \oplus Q_2^t)) \vee (\overline{A} \wedge Q_2^t)$$

En palabras simples: si A es igual a 1, la salida será igual al incremento en una unidad del contador; en otro caso, mantendrá su estado actual. El circuito final, entonces, es el siguiente:



Esto es equivalente a **la implementación de un multiplexor** para cada salida Q_i^{t+1}, donde la señal A se usa para seleccionar entre el valor incrementado o el actual.

Otras implementaciones que se consideran igual de correctas son las siguientes:

- Mantención del estado $Q_2Q_1Q_0 = 111$ una vez que se llega al valor máximo, en vez de dejar que por *overflow* vuelva a ser 000.

Se otorgan **0.5 ptos.** por cada salida correctamente computada en un flanco de subida; y **0.5 ptos.** si, además, se actualiza correctamente según el valor de la señal A. Se considera válido el uso de multiplexores en el circuito. El uso de sumadores, en cambio, **se considera válido si, y solo si se incluyen los diagramas internos de los elementos que lo componen** (*half-adder*, *full-adder* y sus conexiones dentro del sumador). Si se ocupa como caja negra, se otorga como máximo la mitad del puntaje si la implementación es correcta.